

Instruction: OpenFlow

Software-Defined Networking & OpenFlow:

Software-Defined Networking (SDN) is a recently proposed networking paradigm in which the data and the control planes are decoupled from one another. One can think of the control plane as being the network's "brain", i.e., it is responsible for making all decisions, for example, how to forward data, while the data plane is what actually moves the data. In traditional networks, both the control- and data planes are tightly integrated and implemented in the forwarding devices that comprise a network.

The SDN control plane is implemented by the "controller" and the data plane by "switches". The controller acts as the "brain" of the network, and sends commands (a.k.a. "rules") to the switches on how to handle traffic. OpenFlow has emerged as the de facto SDN standard and specifies how the controller and the switches communicate as well as the rules controllers install on switches.

Mininet and OpenFlow:

In homework 1 question 8, we experimented with Mininet using its internal controller. We can also use our own designed controller to send commands to the switches. We will be using the POX controller in this project, which is written in Python.

OpenFlow 1.3 Overview:

OpenFlow 1.3 is the version of the OpenFlow protocol supported within the Mininet environment. The following diagram explains the operation of OpenFlow switches.

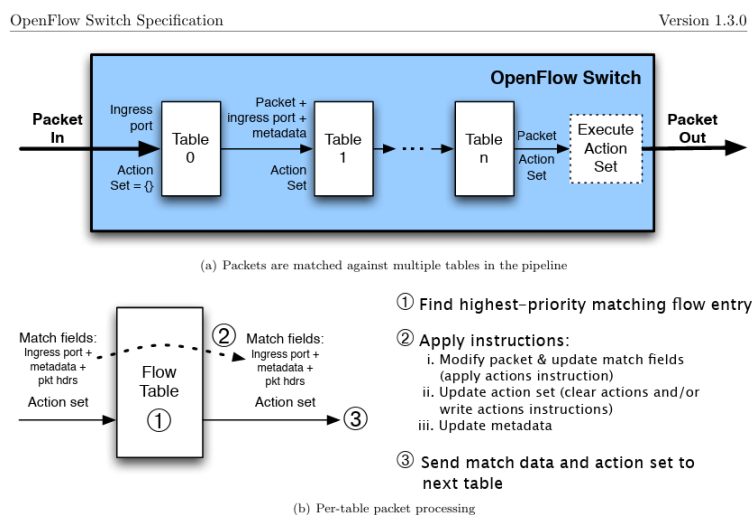


Figure 2: Packet flow through the processing pipeline

Note that when the packet comes into an OpenFlow switch, the switch will reference a table containing “rules” and “actions”. This “flow” table contains the following fields:

OpenFlow Switch Specification

Version 1.3.0

Match Fields	Priority	Counters	Instructions	Timeouts	Cookie
--------------	----------	----------	--------------	----------	--------

Table 1: Main components of a flow entry in a flow table.

The figure below shows the flow of execution that follows. If an `ofp_packet_in` does not match any of the flow entries and the flow table does not have a “table-miss” flow entry, the packet will be dropped. If the packet matches the “table-miss” flow entry, it will be forwarded to the controller. If there is a match-entry for the packet, the switch will execute the action stored in the instruction field of the corresponding flow table.

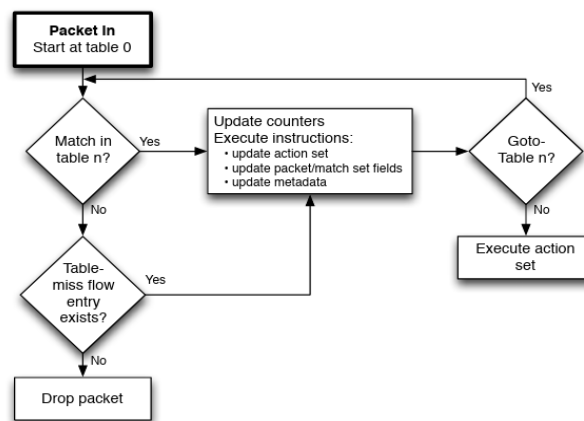


Figure 3: Flowchart detailing packet flow through an OpenFlow switch.

All of the figures and information in this section are from the [OpenFlow 1.3 specification](#), which you can reference if you would like additional information

Running the Code:

To run the controller, place `project3controller.py` in the `~/pox/pox/misc` directory. You can then launch the controller with the command **`sudo ~/pox/pox.py misc.project3controller`**

To run the mininet file, place it in `~` and run the command **`sudo python ~/project3.py`**

To do project3, you will need to run both files at the same time (in 2 different terminal windows).

Useful Resources:

Please check the following document to install and then use `pox` controller (for this project, you should first install POX): http://intronetworks.cs.luc.edu/auxiliary_files/mininet/poxwiki.pdf